# PROYECTOS FINALES

*Persistencia políglota*

Dos enunciados independientes · Guía para grupos de estudiantes

Proyecto A – desarrollar los grupos pares,

Proyecto B – desarrollar los grupos impares

## Propósito académico

Diseñar e implementar un sistema web que resuelva operaciones reales de una organización utilizando persistencia políglota. Los grupos de estudiantes reciben los dominios y sus restricciones, investigan el gestor adecuado para cada uno, implementan sus bases de datos como contenedores Docker y desarrollan una API que permita operar desde la interfaz web y evidenciar qué datos se almacenan, consultan y sincronizan.

## Condiciones generales para ambos grupos

- Equipo sugerido: Grupos de estudiantes que trabajan en clase.
- Cada solución debe utilizar al menos tres tecnologías de persistencia con modelos o propósitos distintos, justificadas por necesidades reales; no se premia usar más gestores sin necesidad.
- Una API REST (monolito modular permitido) constituye el punto de entrada de la web. El navegador no se conecta directamente a las bases de datos.
- Todos los gestores y la API deben ejecutarse en contenedores gestionados mediante Docker Compose; el frontend puede ejecutarse también en un contenedor. Usar volúmenes persistentes y red interna.
- Se exige al menos una interacción entre dominios que atraviese dos tecnologías de persistencia y una sincronización o proyección entre gestores cuando haya datos derivados.
- Emplear datos ficticios; no integrar pagos reales ni información clínica personal. Identificar el Source of Truth de cada dato y justificar consistencia, posibles fallos y recuperación.
- Se permite elegir HTML/CSS/JavaScript, React, Vue, FastAPI, Flask, Express u otra tecnología de desarrollo que el grupo pueda explicar y ejecutar.

## Qué debe decidir el estudiante

| Decisión | Evidencia requerida |
|---|---|
| **Selección de gestores** | Matriz por dominio: patrón de acceso, estructura de datos, consistencia, volumen, disponibilidad, alternativa descartada y justificación. |
| **Modelo de datos** | Entidades/documentos, claves e identificadores compartidos, relaciones, índices, retención y ejemplos JSON/SQL. |
| **Integración** | Diagrama web → API → módulos → gestores; responsables de escritura y lectura; mensajes o sincronizador cuando aplique. |
| **Transacciones** | Delimitar transacción local, validaciones, idempotencia, recuperación y consistencia eventual donde corresponda. |
| **Despliegue** | Docker Compose reproducible, variables de entorno, volúmenes, instrucciones de arranque y datos semilla. |

## Forma de trabajo sugerida

| Fase | Actividades | Producto de control |
|---|---|---|
| **1. Comprensión** | Leer caso, delimitar actores, reglas, historias de usuario y dominios. | Mapa de dominio y flujos. |

| | | |
|---|---|---|
| **2. Investigación** | Comparar candidatos de gestores por dominio con fuentes técnicas y prueba pequeña. | Matriz argumentada de selección. |
| **3. Diseño** | Modelo de datos, API, pantallas, integración y fallos. | Diagramas y contratos de API. |
| **4. Implementación** | Configurar Compose, crear persistencia, endpoints y frontend. | Repositorio ejecutable. |
| **5. Validación** | Pruebas funcionales, integrales, fallos y consistencia. | Resultados y evidencias. |
| **6. Defensa** | Demostración end-to-end, justificar decisiones y responder cambio de requisito. | Exposición y memoria técnica. |

## Arquitectura lógica exigida

Usuario → interfaz web → API REST → módulos de dominio → gestor/es responsable/s. Cuando un dominio mantiene una vista derivada de otro, proponer: escritura en fuente oficial → captura/publicación del cambio → actualización de proyección → consulta de lectura. No suponer transacción ACID conjunta entre gestores.

# PROYECTO A · Clínica de atención ambulatoria

***Caso 1 | Gestión de citas, disponibilidad y búsqueda de servicios***

## 1. Contexto empresarial

Una red de consultorios necesita ofrecer una web donde pacientes ficticios puedan consultar servicios y profesionales, solicitar una cita y seguir su estado. Recepción gestiona agenda, confirmaciones y cancelaciones. El sistema debe evitar reservas duplicadas y permitir búsquedas ágiles por especialidad, horario y ubicación. No se implementará historia clínica ni diagnóstico.

## 2. Usuarios y roles

- Paciente: consulta oferta, solicita/cancela citas y visualiza las propias.
- Recepcionista: gestiona agenda y confirma o reprograma citas.
- Administrador: mantiene profesionales, sedes, especialidades y servicios.

## 3. Dominios de negocio asignados

Los dominios están fijados por el enunciado. El equipo debe elegir y justificar el gestor de cada uno; no se entrega una solución tecnológica predefinida.

| Dominio | Datos y reglas de negocio | Operaciones mínimas |
|---|---|---|
| **Catálogo de servicios y profesionales** | Ficha con atributos variables: especialidad, idiomas, sede, duración, modalidades y horarios de referencia. | Crear, editar, desactivar; visualizar ficha. |
| **Agenda y cupos** | Franjas horarias por profesional y sede; impedir doble reserva concurrente. | Publicar disponibilidad; bloquear/confirmar/liberar cupo. |
| **Citas** | Paciente ficticio, franja, estado y trazabilidad de cambios. | Reservar, consultar, reprogramar, cancelar. |
| **Búsqueda** | Vista optimizada para filtros por especialidad, sede y texto; puede derivarse del catálogo. | Buscar, filtrar y ordenar resultados. |
| **Sesiones y preferencias** | Selección temporal de filtros, sesiones o borradores de reserva; definir vencimiento. | Crear, recuperar y caducar estado temporal. |

## 4. Flujos integrales obligatorios

1. El administrador crea una especialidad, un profesional y un servicio; la web permite encontrarlos mediante búsqueda.

2. Un paciente filtra por especialidad y sede, visualiza franjas y solicita una cita; el sistema registra el estado y reduce la disponibilidad.

3. Dos usuarios intentan reservar simultáneamente la misma franja; solo uno debe obtener confirmación.

4. Recepción reprograma una cita y la web refleja el estado y la nueva disponibilidad.

5. Se cambia un atributo del profesional en la fuente oficial: describir y demostrar cómo se actualiza una vista de búsqueda derivada, si existe.

## 5. Pantallas web mínimas

- Inicio: buscador de servicios/profesionales y filtros.
- Detalle de profesional/servicio y disponibilidad.
- Formulario de reserva con confirmación/error visible.
- Mis citas: estado, cancelación y reprogramación.

- Panel de recepción y administración: mantenimiento y agenda.
- Pantalla técnica opcional: estados del sincronizador, sin exponer credenciales.

## 6. Reglas de arquitectura y consistencia

- La disponibilidad no puede depender únicamente de una caché o índice de búsqueda: la confirmación exige control consistente en el registro autoritativo.
- Distinguir catálogo oficial y proyección de búsqueda cuando se separen; explicar actualización y posible retraso.
- Una reserva debe tener identificador único y prevenir duplicados por reintento.
- La cancelación libera disponibilidad sin perder historial de estado.
- Definir conducta cuando se apague la búsqueda o el servicio de estado temporal; no prometer recuperación instantánea sin mecanismo.

## 7. Pruebas demostrables en la defensa

1. Registrar profesional y comprobar su aparición en la pantalla de búsqueda.

2. Modificar especialidad o sede y medir/reflejar el retraso de actualización de una vista derivada.

3. Intentar dos reservas para un mismo horario y mostrar que no se confirma doble cupo.

4. Cancelar y volver a reservar el horario liberado.

5. Reiniciar contenedores y verificar persistencia de citas; demostrar reacción a indisponibilidad de búsqueda.

## 8. Pregunta de investigación

¿Qué operaciones requieren consistencia fuerte y cuáles pueden tolerar consistencia eventual? Comparen dos alternativas de persistencia para agenda y búsqueda y presenten el motivo por el que descartan una.

# PROYECTO B · Empresa de mensajería urbana

***Caso 2 | Registro de envíos, seguimiento y despacho***

## 1. Contexto empresarial

Una empresa de entregas dentro de la ciudad necesita una web para registrar paquetes, asignarlos a repartidores y ofrecer seguimiento de cada envío. Operadores y repartidores trabajan con estados operativos; clientes ficticios consultan el recorrido y pruebas de entrega. Se utilizarán coordenadas y destinatarios inventados; no hace falta integrar mapas comerciales ni GPS real.

## 2. Usuarios y roles

- Cliente: crea una solicitud de envío y consulta su seguimiento.
- Operador: valida solicitudes, asigna repartidores y gestiona incidencias.
- Repartidor: acepta asignación, reporta estados y registra evidencia simulada.
- Administrador: mantiene zonas, tipos de servicio y capacidad operativa.

## 3. Dominios de negocio asignados

Los dominios están fijados por el enunciado. El equipo debe elegir y justificar el gestor de cada uno; no se entrega una solución tecnológica predefinida.

| Dominio | Datos y reglas de negocio | Operaciones mínimas |
|---|---|---|
| **Solicitudes y envíos** | Origen/destino ficticios, remitente, destinatario, tipo de servicio, código único y estado vigente. | Crear, aceptar, consultar y cancelar según regla. |
| **Despacho y asignaciones** | Repartidor, disponibilidad, carga actual y vínculo único con envío activo. | Asignar, reasignar, liberar capacidad. |
| **Seguimiento y eventos** | Secuencia temporal de cambios de estado y ubicación simulada; conservar orden y autor. | Registrar retiro, tránsito, incidencia, entrega; consultar línea de tiempo. |
| **Evidencias de entrega** | Datos heterogéneos: nombre del receptor, observación, foto simulada mediante URL/local o referencia de archivo. | Registrar y consultar prueba de entrega. |
| **Consulta operativa** | Búsqueda de envíos por código, zona, estado, destinatario ficticio y fecha. | Buscar y filtrar rápidamente; decidir si se requiere índice derivado. |
| **Estado temporal** | Última posición simulada/estado de conexión de repartidores con expiración definida. | Actualizar, consultar y marcar dato no vigente. |

## 4. Flujos integrales obligatorios

1. Cliente registra un envío; la API asigna código único y presenta el comprobante en web.

2. Operador encuentra pendientes y asigna un repartidor disponible; el sistema evita una asignación que exceda su capacidad configurada.

3. Repartidor simula retiro, tránsito e incidencia; el cliente visualiza el historial de eventos.

4. El repartidor confirma entrega y registra evidencia; la web conserva estado actual e historial sin contradicciones.

5. Se modifica la información que alimenta una vista operativa o de búsqueda y se demuestra su sincronización, si el grupo la separó del dato oficial.

## 5. Pantallas web mínimas

- Formulario de solicitud y pantalla de confirmación.
- Consulta de seguimiento por código con línea de tiempo.
- Panel de despacho: lista, filtros y asignación.
- Vista de repartidor: tareas y botones de cambio de estado.
- Detalle de envío: estados y evidencia de entrega.
- Panel administrativo: zonas, repartidores y capacidades.

## 6. Reglas de arquitectura y consistencia

- Establecer una máquina de estados válida, por ejemplo creado → asignado → retirado → en tránsito → entregado; incidencias y cancelación con reglas explícitas.
- El estado actual no debe contradecir el historial; identificar cuál es autoritativo y cómo reconstruir o reconciliar el otro.
- No asumir que ubicación temporal equivale a prueba de entrega; la evidencia confirmada debe conservarse.
- Los cambios de estado deben ser idempotentes ante reintentos y registrar fecha, actor e identificador de envío.
- La búsqueda es una proyección opcional: si falla, el registro de entrega debe poder seguir operando según la arquitectura propuesta.

## 7. Pruebas demostrables en la defensa

1. Crear envío y demostrar código único y consulta posterior.

2. Simular secuencia válida y rechazar una transición inválida, por ejemplo creado → entregado sin pasos autorizados.

3. Intentar dos asignaciones incompatibles y comprobar control de capacidad.

4. Reiniciar el contenedor de estado temporal y distinguir información efímera de historial duradero.

5. Demostrar que la entrega registrada permanece tras reinicio; provocar caída del mecanismo de búsqueda/sincronización y explicar recuperación.

## 8. Pregunta de investigación

¿Conviene guardar seguimiento como registros relacionales, documentos o eventos inmutables? Analicen consultas dominantes, orden, auditoría, costes operativos y recuperación del estado actual.

## 9. Contrato mínimo de interfaz y API (aplicable a cada caso)

El equipo define nombres definitivos y documenta request/response, validaciones y códigos HTTP. La siguiente matriz indica los comportamientos, no una tecnología obligatoria.

| Categoría | Ejemplos orientativos | Demostración |
|---|---|---|
| **Lectura** | GET /servicios, /profesionales, /envios/{codigo}, /citas/{id} | Mostrar información proveniente de gestores. |
| **Escritura** | POST /citas, /envios, /asignaciones; PATCH /citas/{id}, /envios/{id}/estado | Ejecutar validaciones y confirmar resultados en web. |
| **Búsqueda** | GET /buscar?... o GET /envios?estado=... | Filtrar y explicar origen de datos e índices. |
| **Salud** | GET /health (sin publicar secretos) | Probar disponibilidad de API/dependencias. |

## 10. Entregables obligatorios

1. Repositorio con README: requisitos previos, ejecución limpia, credenciales ficticias, URLs, reinicio y limpieza sin pérdida involuntaria de datos.

2. docker-compose.yml y Dockerfiles necesarios; al menos tres motores de persistencia, API y opcionalmente interfaz, con volúmenes y variables configurables.

3. Informe técnico: contexto, mapa de dominios, análisis comparativo de gestores (al menos dos opciones por dominio relevante), selección, modelo de datos, índices y decisiones de Source of Truth.

4. Diagrama de arquitectura, diagrama de dos flujos end-to-end y tabla de consistencia/errores entre dominios.

5. Especificación de API (OpenAPI o tabla completa con payload y códigos).

6. Web operativa con formularios reales, consulta de resultados y mensajes de éxito/error. No se acepta únicamente maqueta visual.

7. Datos semilla de demostración y colección de pruebas automatizadas o instrucciones de prueba reproducibles.

8. Registro de evidencias: capturas o breve video de operaciones y pruebas de fallos; reflexión sobre límites y mejoras futuras.

## 11. Guion de defensa (15–20 minutos por grupo)

1. Explicar el problema y los dominios asignados (2 min).

2. Mostrar matriz comparativa y justificar cada gestor, incluso uno descartado (4 min).

3. Ejecutar docker compose up --build -d y mostrar los servicios (2 min; preparación previa permitida si las imágenes son grandes).

4. Demostrar dos flujos completos mediante la web, comprobando cambios en gestores (6 min).

5. Provocar una situación de fallo o concurrencia, explicar consistencia y recuperación (4 min).

6. Responder cambio imprevisto de requisito y señalar límites de la solución (2 min).

## 12. Restricciones de integridad y seguridad del laboratorio

- Datos personales, coordenadas y evidencias exclusivamente ficticios. Evitar exponer puertos de las bases de datos más allá del equipo de práctica.
- No colocar contraseñas en frontend ni registrar secretos en el repositorio. Usar .env.example y mecanismos sencillos de configuración.
- La API valida tipos, estados y autorizaciones; no confiar en precios, horarios, capacidades o estados enviados por el navegador.
- Si se usa caché o motor de búsqueda, no considerarlo fuente única para una operación que requiera confirmación consistente.

- Registrar errores y presentar respuesta clara al usuario; documentar cuándo una operación puede ser reintentada con seguridad.